

StreamHive Platform End-to-End Architecture & Implementation Report

1. System Overview

The StreamHive website is built on a modern, cloud native microservices architecture that employs several reliability and scalability patterns. The system is engineered to support multiple concurrent users, enabling the utilization of the platform with videos of varying quality and resolutions while ensuring consistency and synchronization across distributed services.

This platform delivers a resilient, observable, and scalable auction system using:

- Microservices + Event-Driven Architecture (NATS): loose coupling, async workflows.
- DB per Service + CQRS: independent scaling, optimized reads.
- Resilience Patterns: Bulkhead, Circuit Breaker, Retry/Backoff, Timeouts.
- Distributed Transactions: Choreography-based Sagas with compensation.
- Consistency: Clear models (strong/eventual), dual-write avoidance tactics.
- Observability: Prometheus → Grafana (metrics).
- Container/ Orchestrator: Docker, Kubernetes, and Helm Chart
- Platform: Azure Kubernetes Service (AKS) with Service Mesh, API Gateway, CI via GitHub Actions, GitOps via ArgoCD.
- Security: Service-to-service auth (JWT), API authorization.
- Cloud Infrastructure: Terraform
- Continuous Integration (CI): GitHub Action / Jenkins
- Continuous Deployment (CD): ArgoCD
- Version Control: Git

StreamHive is a video streaming app that allows users to watch, share, and store their favorite videos on one simple platform.

Key Features:

- **Secure User Accounts:** A strong login system keeps user details and uploaded videos safe and private.
- **Private Video Uploading:** Users can upload videos for personal storage or share them securely with others.
- **User-Controlled Content Management:** Users can manage their videos easily and delete them permanently at any time.
- **Flexible Video Quality Control:** Users can adjust video quality based on their internet speed, from low resolution to high definition.

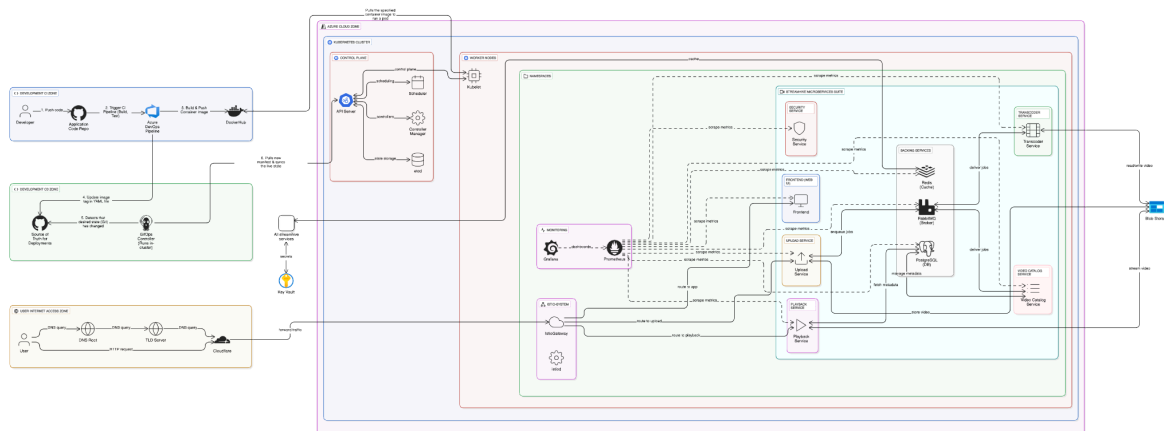
2. Architecture

2.1 Logical View

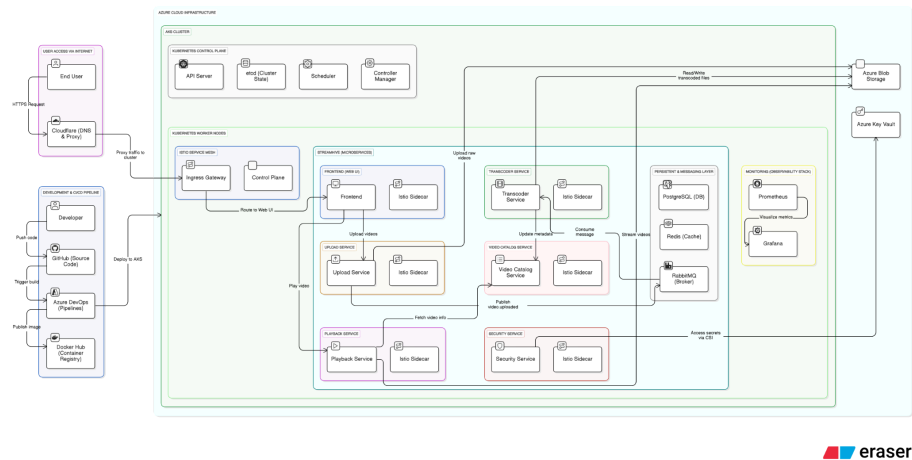
- **Edge:** Cloudflare is used as the primary API Gateway and reverse proxy. It handles initial user requests and provides critical services like rate limiting, DDoS protection, caching static content (CDN), and securely routing traffic to the cluster's entry point.
- **Mesh:** Istio Service Mesh manages all service-to-service communication within the Kubernetes cluster. It provides automatic mTLS encryption for security, intelligent routing (traffic policies), resiliency patterns like retries and timeouts, and generates detailed telemetry (metrics, logs, traces) for every request.
- **Services:** Frontend(Web UI), Security, Upload, Transcoder, Video Catalog, Playback, OnboardingOrchestrator.
- **Data:** One database per service - PostgreSQL , Azure Blob Storage, Redis.
- **Events:** Rabbitmq broker for enabling resilient, asynchronous workflows.
- **Observability:** Istio (metrics, logs), Prometheus, Grafana.
- **Ops:** Azure DevOps, ArgoCD, Cluster Autoscaler.

StreamHive Logical Architecture Diagram

<https://app.eraser.io/workspace/j5RSQb8mqDtCia0wPYXF?origin=share>



2.2 High-Level Architecture

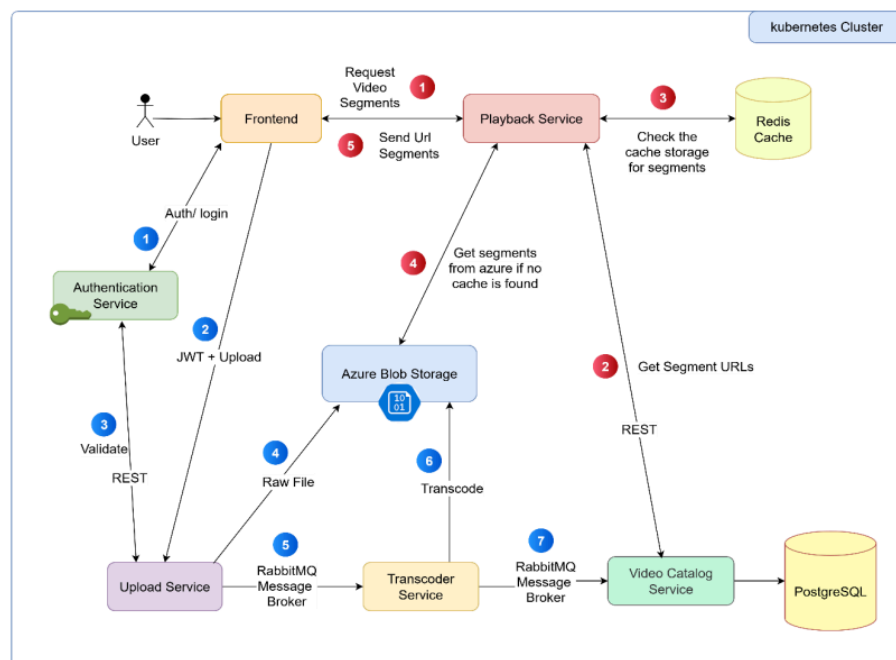


StreamHive Logical Architecture Diagram

<https://app.eraser.io/workspace/HKdPkOQLFVJRq1ZdnXBm?origin=share>

Architecture Diagram

This diagram illustrates the flow: User requests video segments, Frontend interacts with Playback Service, which checks Redis Cache for segments. If not found, it gets segments from Azure Blob Storage. Authentication Service validates user login via JWT. Upload Service handles raw file upload, which then goes to Transcoder Service via RabbitMQ. Transcoder converts raw video into multiple resolutions. Video Catalog Service manages metadata in PostgreSQL.



3. Service Architecture & Patterns

Microservices & DB-Per-Service

- Isolation: Each service owns its data schema and lifecycle; no cross-service DB access.
- Benefits: Independent deployments, targeted scaling, blast radius control, schema autonomy.
- Contract: Interact via APIs + Events (never shared tables).

3.1 Service Communication

Services communicate primarily through:

- Synchronous REST APIs for direct queries
- Asynchronous events for state propagation and process coordination

Event-Driven Communication

The system heavily uses event-driven architecture with NATS Streaming as the central event

bus:

Service A → Event Published → NATS → Event Consumed → Service B

Event-Driven Architecture (EDA)

- Choreography: Services publish/subscribe to domain events; no central coordinator for simple flows.
- Delivery: At least once; consumers must be idempotent.
- Event Contracts: Versioned schemas, backward compatible, include correlation & causation IDs.

Services Breakdown

- Authentication Service: Manages user registration, login, and logout using JWT-based authentication.
- Upload Service: Manages the secure ingestion and initial storage of raw video files uploaded by users.
- Transcoder Service: Transcodes raw video into multiple resolutions (1080p, 720p, etc.) using the FFmpeg library, necessary for adaptive bitrate streaming (HLS).
- Video Catalog Service: Manages all video metadata, including titles, descriptions, and user-defined privacy settings.
- Playback Service: Delivers video content to users for playback, working with transcoded video files to automatically adjust video quality based on user settings and network conditions.

3.2 Implementation Steps

- Build Pipeline: All repositories were hosted in GitHub. Azure DevOps pipeline was triggered to build Docker images from repositories and upload them to Docker Hub. Azure DevOps pipeline build agent was manually configured using Azure VM.
- Deployment Pipeline: Deployment Pipeline pulls images from Docker Hub and applies deployments to the Kubernetes cluster.
- Kubernetes Cluster: Due to Azure constraints, AKS was not applicable. Two standard Azure VMs were used to manually configure a Kubernetes cluster (1 VM for master node, 1 VM for worker node). Kubernetes cluster secrets such as Azure Blob Storage URLs were pulled securely using Azure Key Vault.
- Microservices: Istio service mesh was used for the API gateway. RabbitMQ queues were used for asynchronous inter-service communication. Redis cache was implemented in the Playback service to avoid repeatedly downloading from Azure storage.
- Frontend: The Frontend application was developed using React. To avoid high resource usage in the cluster, the application was built using Vite to serve static pages. Nginx was used to serve these pages.
- Security Design: JWT-Based authentication is used to authenticate users.